The Complete Agent Development Guide

Contents

1 Overview **2**

2 The Industrialization Journey at a Glance **3**

3 1. Build Locally with Microsoft Agent Framework v1.0 **3**
3.1 What's stable in v1.0 . . . 3
3.2 Hello agent (Python) . . . 4
3.3 Foundry Toolkit for VS Code . . . 4

4 2. Agent Harness — Shell, Files, and Long Sessions **4**
4.1 2.1 Local Shell Harness with approval flows . . . 5
4.2 2.2 Hosted Shell Harness — same code, sandbox runtime . . . 5
4.3 2.3 Context Compaction for long-running sessions . . . 5
4.4 2.4 Multi-agent composition with the GitHub Copilot SDK . . . 6

5 3. Make Agents Stateful — Memory in Foundry **6**
5.1 What's new in this refresh . . . 6
5.2 Pricing (consumption begins June 1, 2026) . . . 7

6 4. Toolbox in Foundry — One Endpoint, Any Agent **7**
6.1 The problem Toolbox solves . . . 7
6.2 The four pillars . . . 7
6.3 Anatomy . . . 8
6.4 Endpoint shape . . . 9
6.5 Build a toolbox (Python) . . . 9
6.6 Consume from any framework . . . 9
6.7 Promote to default . . . 10

7 5. Hosted Agents in Foundry Agent Service **10**
7.1 Why traditional compute doesn't fit agents . . . 10
7.2 Core capabilities (2026 refresh) . . . 11
7.3 Container model . . . 11
7.4 Identities involved . . . 11
7.5 Sessions vs Conversations . . . 12
7.6 Choosing a protocol . . . 12
7.7 Versioning, canary, and blue/green . . . 13
7.8 One-command deployment . . . 13

8 6. End-to-End Reference Architecture **14**
8.1 Layer responsibilities . . . 14

9 7. Industrialization with `azd` and CI/CD **14**
9.1 Repository layout . . . 14
9.2 Key pieces of `agent.yaml` . . . 15
9.3 Promotion workflow (GitHub Actions) . . . 15
9.4 One-command developer flow . . . 15

10 8. Security & Governance **16**

10.1 Identity & access (no shared service accounts) . . . 16
10.2 Network isolation . . . 16
10.3 Content safety & DLP . . . 16
10.4 Secrets . . . 17
10.5 Third-party tool risk . . . 17

11 9. Observability & Operations 17
11.1 What's in scope . . . 17
11.2 Recommended baseline alerts . . . 18

12 10. Cost & FinOps 18
12.1 Memory pricing reminder (effective June 1, 2026) . . . 18
12.2 Patterns to adopt early . . . 18

13 11. Decision Matrix — Hosted Agent vs Prompt-Based Agent 19

14 12. Test & Simulation 19
14.1 Inject failures at the protocol layer . . . 19
14.2 Validate canary metrics with KQL . . . 20
14.3 Adversarial coverage . . . 20
14.4 Chaos for downstream services . . . 20

15 13. Production Readiness Checklist 20
15.1 Architecture . . . 20
15.2 Identity & Security . . . 20
15.3 Tools . . . 20
15.4 State . . . 21
15.5 Operations . . . 21
15.6 Distribution . . . 21

16 14. Required Personas for Industrialization 21

17 15. References 22
17.1 Source articles (this guide synthesizes and extends them) . . . 22
17.2 Microsoft Learn . . . 22
17.3 Tooling . . . 22
17.4 Community . . . 22

Industrializing AI agents on Microsoft Foundry — from local prototype to production-grade enterprise deployment.

---

1 Overview

Building a working agent on a developer laptop is no longer the hard part. The hard part is taking that same agent and operating it for **thousands of users**, with **per-session isolation**, **enterprise identity**, **governed tools**, **persistent memory**, **observability**, and **scale-to-zero economics** — without rebuilding the platform every time.

Microsoft Foundry now offers an end-to-end story for that journey, structured around five capabilities that landed (or stabilized) together:

Capability	Status	What it changes
Microsoft Agent Framework v1.0	**GA**	Stable Python + .NET runtime that unifies Semantic Kernel & AutoGen
Foundry Toolkit for VS Code	**GA**	Local create / debug / deploy loop with traces
Memory in Foundry Agent Service	**Public Preview**	Managed long-term memory, no DB to provision
Toolbox in Foundry	**Public Preview**	One MCP endpoint exposing many tools, governed centrally
Hosted Agents in Foundry Agent Service (refresh)	**Public Preview**	Per-session VM-isolated sandbox, scale-to-zero, persistent FS, per-agent Entra ID
Observability in Foundry Control Plane	**GA on core**	End-to-end OpenTelemetry tracing, Red Teaming Agent, continuous evaluation

This guide walks through the full industrialization pipeline, with code samples, an end-to-end reference architecture, governance and FinOps controls, and a production checklist.

---

2 The Industrialization Journey at a Glance

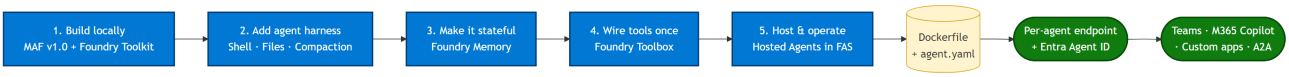

Figure 1: Diagram 1

Each step is **independent and reversible** — you can adopt them in any order — but they reinforce each other and are designed around a single deployment artifact: a container image plus an `agent.yaml` declaration deployable in one `azd deploy`.

---

3 1. Build Locally with Microsoft Agent Framework v1.0

3.1 What's stable in v1.0

Microsoft Agent Framework (MAF) is the open-source SDK and runtime that unifies the enterprise foundations of **Semantic Kernel** with the multi-agent orchestration patterns of **AutoGen**. Migration guides are published for both predecessors so existing investments carry forward.

Pillar	What you get
Multi-model	Azure OpenAI, Anthropic, Google Gemini, Amazon Bedrock, Ollama, OpenAI direct, and more
Workflows	Programmatic *and* declarative multi-step pipelines, visualizable in DevUI
Native Foundry integration	Memory, hosted agents, Observability, and Foundry Tools as first-class building blocks
Open standards	MCP, A2A, OpenAPI built in
Languages	Python and .NET, identical concepts

3.2 Hello agent (Python)

```python
import asyncio
from agent_framework.azure import AzureOpenAIResponsesClient
from azure.identity import AzureCliCredential

async def main():
    agent = AzureOpenAIResponsesClient(
        credential=AzureCliCredential(),
    ).as_agent(
        name="SupportTriageBot",
        instructions="You are an expert support triage agent.",
    )
    response = await agent.run("Analyze this ticket and classify its priority.")
    print(response.text)

asyncio.run(main())
```

3.3 Foundry Toolkit for VS Code

The Toolkit (formerly *AI Toolkit for VS Code*) is now **GA** and gives you:

- Project scaffolding from templates or from GitHub Copilot prompts
- Local run + step-through debugging with **trace visualization**
- Direct `Deploy to Foundry Agent Service` action — no leaving VS Code
- Pre-release channel for the newest Hosted Agents experience

This is the recommended inner loop for any agent destined for Foundry hosting.

---

4 2. Agent Harness — Shell, Files, and Long Sessions

Orchestrating *multiple* agents is half the picture; the other half is what one agent does autonomously over a long task: run shell commands, read/write files, manage context across hours of work without losing coherence. MAF ships three foundational harness patterns.

4.1 2.1 Local Shell Harness with approval flows

```
@tool(approval_mode="always_require")
def run_bash(command: str) -> str:
    """Execute a shell command and return the result."""
    result = subprocess.run(command, shell=True, capture_output=True,
 ↪  text=True)
    return result.stdout

local_shell_tool = client.get_shell_tool(func=run_bash)
agent = Agent(
    client=client,
    instructions="You are a helpful assistant that can run shell commands.",
    tools=[local_shell_tool],
)
```

 Always run local shell execution in an isolated environment with approval enabled.

4.2 2.2 Hosted Shell Harness — same code, sandbox runtime

```
shell_tool = client.get_shell_tool()  # execution runs in provider-managed
 ↪  sandbox
agent = Agent(client=client, instructions="...", tools=shell_tool)
```

A **single-line change** moves command execution from the laptop to the VM-isolated sandbox that powers Hosted Agents — no Dockerfile changes, no infrastructure rewiring.

4.3 2.3 Context Compaction for long-running sessions

```
from agent_framework import (
    Agent, InMemoryHistoryProvider,
    CompactionProvider, SlidingWindowStrategy,
)

agent = Agent(
    client=client,
    instructions="You are a helpful assistant.",
    tools=[get_weather],
    context_providers=[
        InMemoryHistoryProvider(),
        CompactionProvider(
            before_strategy=SlidingWindowStrategy(keep_last_groups=3)
        ),
    ],
)
```

Keeps the agent inside its token budget while preserving the most recent meaningful conversation groups — essential for sessions that run for hours or days.

4.4 2.4 Multi-agent composition with the GitHub Copilot SDK

MAF acts as the **orchestration backbone** while delegating heavy code-related work (MCP tools, shell, file ops) to a Copilot-SDK-powered sub-agent. Sequential, concurrent, hand-off, and coding agent patterns are all supported, with checkpointing, middleware pipelines, and human-in-the-loop control built into the execution layer.

---

5 3. Make Agents Stateful — Memory in Foundry

Production agents need to remember user preferences, prior context, and domain facts across sessions — without forcing every team to provision their own database.

```
from agent_framework import InMemoryHistoryProvider
from agent_framework.azure import AzureOpenAIResponsesClient,
 ↪  FoundryMemoryProvider
from azure.ai.projects import AIProjectClient
from azure.identity import AzureCliCredential
import os

project_client = AIProjectClient(
    endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=AzureCliCredential(),
)

agent = AzureOpenAIResponsesClient(
    credential=AzureCliCredential(),
).as_agent(
    name="CustomerSuccessAgent",
    instructions="You are a proactive customer success agent.",
    context_providers=[
        InMemoryHistoryProvider(),
        FoundryMemoryProvider(
            project_client=project_client,
            memory_store_name=memory_store.name,
            scope="user_123",  # segments memory per user
        ),
    ],
)
```

5.1 What's new in this refresh

Capability	Why it matters
Native MAF & LangGraph integration	No middleware glue — memory works out of the box in either framework
Memory item CRUD API	Inspect, edit, delete specific facts the agent stored — full transparency for users and DPO requests

Capability	Why it matters
Custom user scope header	Segment memory by your own `userId`, not necessarily Entra — works with any IAM system

5.2 Pricing (consumption begins June 1, 2026)

Component	Price
Short-term memory	$0.25 per 1K events stored
Long-term memory	$0.25 per 1K memories per month
Memory retrieval	$0.50 per 1K retrievals

Memory is **free during preview** before billing begins.

---

6 4. Toolbox in Foundry — One Endpoint, Any Agent

6.1 The problem Toolbox solves

A single onboarding agent might call five tools (Entra account creation, GitHub repo access, cloud provisioning, ADO tasks, Teams welcome). That means **five auth models, five owning teams, five integration codebases**. Multiply by every agent in the org and you get duplicated credentials, inconsistent governance, and zero discoverability.

6.2 The four pillars

Pillar	Available	What you get
Discover	Coming soon	Find approved tools without browsing catalogs
Build	**Today**	Curate tools into a named, versioned, reusable bundle
Consume	**Today**	One MCP-compatible endpoint for the whole bundle
Govern	Coming soon	Centralized auth + observability across every tool call

6.3 Anatomy

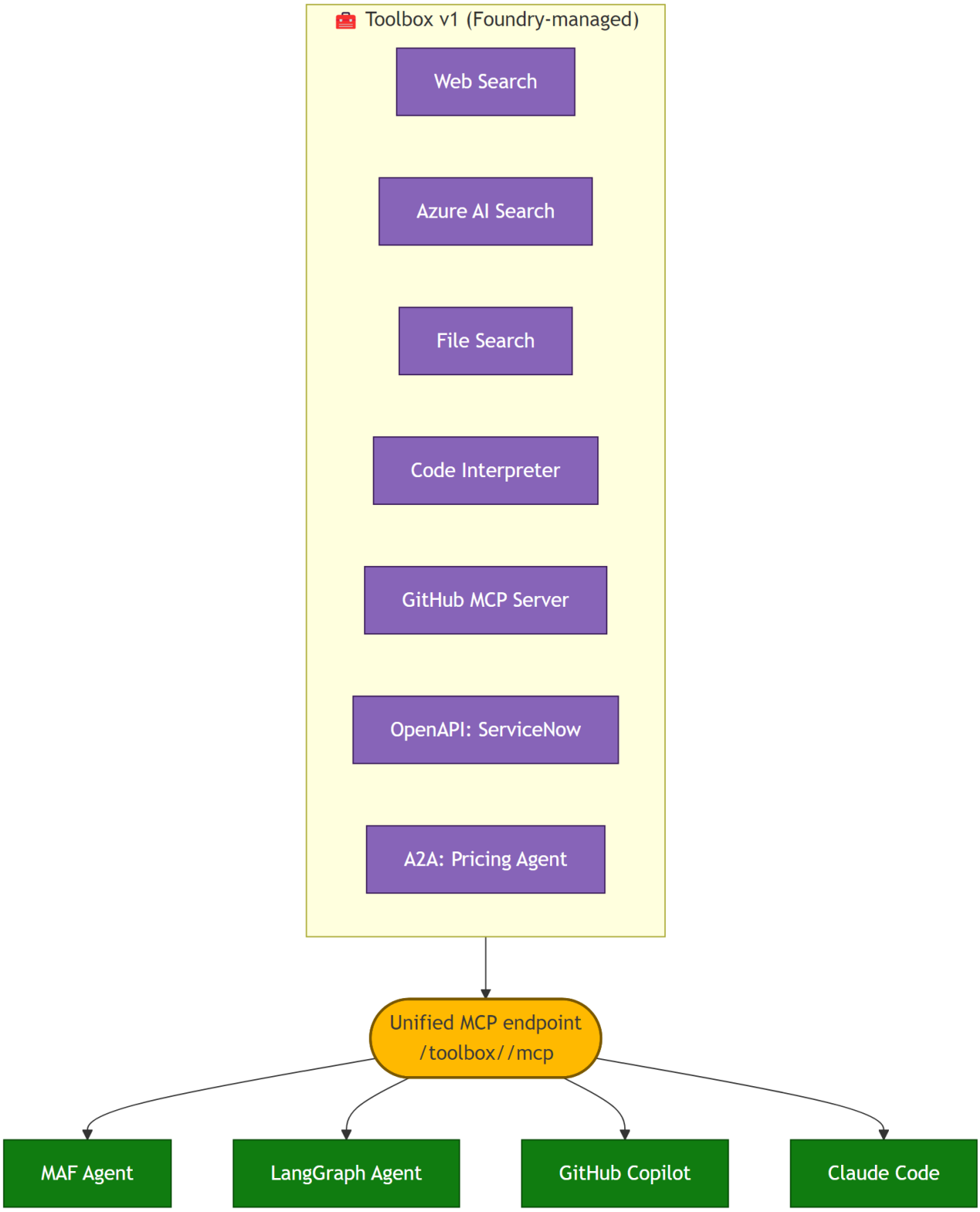

Figure 2: Diagram 2

6.4 Endpoint shape

`https://<resource>.services.ai.azure.com/api/projects/<project>/toolbox/<toolbox-name>/mcp?api-version=v1`

A toolbox is **versioned**: you can iterate freely on a `v2` while consumers stay pinned to `v1`. Promote a version to be the default once it's production-ready, and existing consumers automatically pick it up at the unversioned endpoint.

6.5 Build a toolbox (Python)

```
toolbox_version = client.beta.toolboxes.create_toolbox_version(
    toolbox_name="customer-feedback-triaging-toolbox",
    description="Search docs and triage feedback; act in GitHub.",
    tools=[
        {
            "type": "web_search",
            "description": "Retrieve public docs on official websites",
            "custom_search_configuration": {
                "project_connection_id": "<BING_CONNECTION>",
                "instance_name": "<BING_INSTANCE>"
            }
        },
        {
            "type": "azure_ai_search",
            "name": "product-manuals-ai-search",
            "description": "Search internal product documentation",
            "azure_ai_search": {
                "indexes": [
                    {"index_name": "<INDEX>",
                     "project_connection_id": "<CONNECTION>"}
                ]
            }
        },
        {
            "type": "mcp",
            "server_label": "github",
            "server_url": "https://api.githubcopilot.com/mcp",
            "project_connection_id": "<CONNECTION>"
        }
    ],
)
print(toolbox_version.mcp_endpoint)
```

6.6 Consume from any framework

```
mcp_tool = MCPStreamableHTTPTool(
    name="toolbox",
    url=TOOLBOX_ENDPOINT,
```

```
    http_client=httpx.AsyncClient(
        auth=_ToolboxAuth(token_provider),  # injects Entra bearer per request
        headers={"Foundry-Features": "Toolboxes=V1Preview"},
        timeout=120.0,
    ),
    load_prompts=False,
)
agent = chat_client.as_agent(
    name=AGENT_NAME,
    instructions=SYSTEM_PROMPT,
    tools=[mcp_tool],
)
```

The agent **discovers all bundled tools dynamically** at runtime — no per-tool wiring, no custom SDK per integration. Authentication options include **OAuth identity passthrough** and **Microsoft Entra Agent Identity**.

> **Foundry-homed, not Foundry-bound.** Toolboxes are configured and governed in Foundry, but consumed by any MCP-capable runtime — Microsoft Agent Framework, LangGraph, custom Python, **GitHub Copilot, Claude Code, MCP-enabled IDEs**, and so on.

6.7 Promote to default

```
client.beta.toolboxes.update(
    toolbox_name="customer-feedback-triaging-toolbox",
    default_version="<version_id>",
)
```

---

7 5. Hosted Agents in Foundry Agent Service

This is the production runtime. The 2026 refresh is **fundamentally different** from the original Ignite preview — it now provides **per-session VM-isolated sandboxes**, **persistent filesystem**, **scale-to-zero with state resume**, and **dedicated Entra Agent IDs** out of the box.

7.1 Why traditional compute doesn't fit agents

Containers, App Service, serverless functions were designed for **shared multi-tenant web traffic**. Agents have a fundamentally different access pattern:

Concern	Traditional compute	Hosted Agents
Isolation	Multiple sessions share a container	Every session gets a dedicated VM-isolated sandbox
Cold start	Seconds to minutes (high variance)	Seconds (low variance, predictable)

Concern	Traditional compute	Hosted Agents
Idle cost	Always-on billing or slow scale-from-zero	Scale-to-zero with FS-preserving resume
State persistence	You build it (DBs, blob, ...)	Built in — files & disk survive scale-to-zero
Identity	Shared service account	Per-agent Entra ID + per-user (OBO)
Observability	You build it	Built in (agent / session / fleet)

7.2 Core capabilities (2026 refresh)

- **Predictable cold starts** — sessions spin up in **<100 ms** in your custom Docker environment
- **Scale to zero** — pay nothing while idle; agents suspend between turns
- **Resume with filesystem intact** — `$HOME` and uploaded `/files` survive scale-to-zero events
- **Per-session isolation** — every session gets a dedicated, hypervisor-isolated sandbox; isolation keys to namespace end-user sessions
- **BYO VNet** — pin outbound traffic through your own virtual network for private connectivity
- **Production endpoints** — built-in version management with **stable endpoints** + **weighted roll-out** between versions
- **Multiple protocols** — Responses (OpenAI-compatible) + Activity (Teams/M365) + Invocations (custom JSON / AG-UI / webhooks) + A2A — all combinable in one agent

7.3 Container model

You package your agent as a **container image**, push it to **Azure Container Registry**, and reference it from `agent.yaml`. The platform pulls the image, provisions compute on demand, assigns a per-agent **Entra Agent ID**, and exposes a dedicated endpoint:

```
Responses   : {project_endpoint}/agents/{name}/endpoint/protocols/openai/v1/responses
Invocations : {project_endpoint}/agents/{name}/endpoint/protocols/invocations
```

Active endpoints depend on the protocols declared in the version definition (`agent.yaml` when using `azd`, or `container_protocol_versions` via the SDK).

7.4 Identities involved

Identity	Scope	Purpose
Agent Entra identity (per agent)	Created automatically at deploy	Runtime auth — model invocation, tool access, downstream Azure
Project managed identity (per project)	System-assigned on the Foundry project	Platform infrastructure ops (e.g., ACR Repository Reader)

When you deploy with `azd`, the **Azure AI User** role is assigned automatically to the agent's Entra identity at account scope. For **external resources** (your own Storage, Cosmos, Key Vault), you assign RBAC manually to that same agent identity.

7.5 Sessions vs Conversations

This is a critical distinction. They serve different concerns and behave differently per protocol.

Concept	What it is	Where it lives	Lifetime
Session	Logical session with persisted compute state — `$HOME`, uploaded files	Tied to compute (provisioned/deprovisioned automatically)	Up to **30 days**; idle timeout **15 min**
Conversation	Durable record of messages, tool calls, responses	Stored in Foundry; independent of compute	As long as you keep it

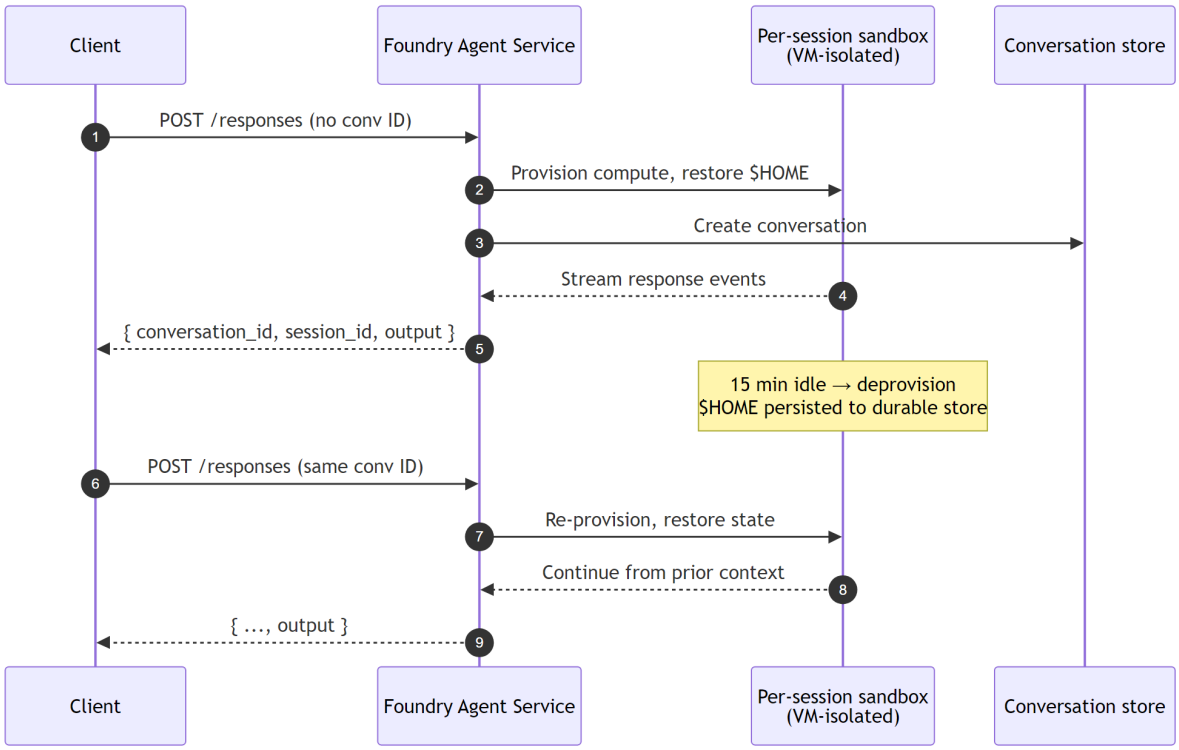

Figure 3: Diagram 3

Responses protocol — conversation ID is primary; the platform manages history; session ID is returned for `/files` uploads. **Invocations protocol** — session ID is primary; you manage state; no platform conversation history.

7.6 Choosing a protocol

Scenario	Protocol	Reason
Conversational chatbot or assistant	**Responses**	Platform-managed history, streaming, lifecycle
Multi-turn Q&A with RAG or tools	**Responses**	Built-in conversation threading + tool result handling
Background / async processing	**Responses**	`background: true` with platform-managed polling
Agent published to Teams or M365	**Responses + Activity**	Responses for logic; Activity for the channel
Webhook receiver (GitHub, Stripe, Jira)	**Invocations**	Caller's payload format, can't be reshaped
Classification / extraction / batch	**Invocations**	Structured-data in / structured-data out
Custom streaming protocol (AG-UI)	**Invocations**	Raw SSE control needed
Protocol bridge / proprietary clients	**Invocations**	Caller defines the contract

 Not sure? Start with **Responses**. You can always add an Invocations endpoint later in the same agent.

7.7 Versioning, canary, and blue/green

Every `create version` produces an **immutable agent version** — a snapshot of the image, resources, env vars, and protocol config. Deployments reference a specific version. To update, you create a new version; to roll out gradually, you split traffic.

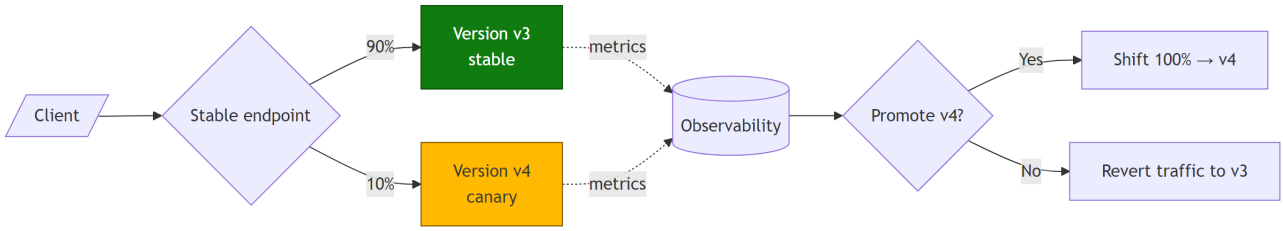

Figure 4: Diagram 4

7.8 One-command deployment

```
azd deploy
```

That single command: - Builds the container image and pushes to ACR - Creates a new immutable agent version - Provisions the per-agent Entra identity - Assigns the Azure AI User role - Wires up the stable endpoint - Configures autoscaling and the CI/CD promotion gates declared in `azd-pipelines.yaml`

---

8 6. End-to-End Reference Architecture

A drawio version of the same architecture — using Azure & Foundry icons — is available at `drawio/Foundry_Agent_Reference_Architecture.drawio`. The mermaid view below captures the same flow for in-document reading.

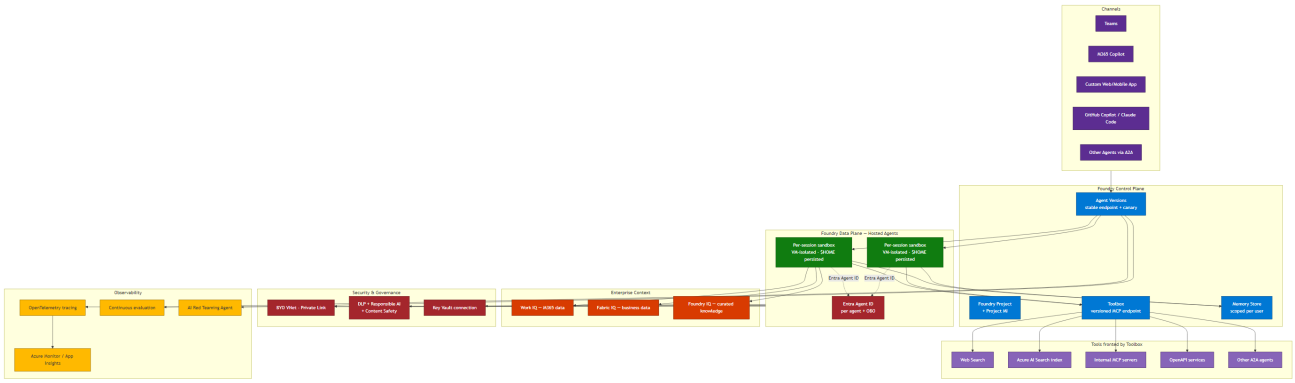

Figure 5: Diagram 5

8.1 Layer responsibilities

Layer	What it owns
Channels	User-facing entry points — published once, consumed many ways
Foundry Control Plane	Project, agent registry/versions, Toolbox catalog, Memory store config
Hosted Agents (data plane)	Per-session VM-isolated sandboxes with `$HOME` persistence + per-agent Entra ID
Toolbox	Single MCP endpoint exposing curated, governed, multi-protocol tools
Enterprise Context	Work IQ (M365), Fabric IQ (business data), Foundry IQ (curated knowledge & search)
Security & Governance	BYO VNet, DLP, Responsible AI, Content Safety, Key Vault
Observability	OpenTelemetry, evaluation, red-teaming, Azure Monitor / App Insights

---

9 7. Industrialization with `azd` and CI/CD

9.1 Repository layout

```
my-agent/
├── src/
│   ├── agent.py              # MAF / LangGraph code
```

```
│   ├── responses_handler.py # Responses protocol entrypoint
│   └── tools/               # local tool wrappers
├── Dockerfile               # custom runtime + dependencies
├── agent.yaml               # protocols, resources, env vars, version
├── azure.yaml               # azd service mapping
└── infra/
    ├── main.bicep           # Foundry project, ACR, VNet, RBAC
    └── modules/
```

9.2 Key pieces of `agent.yaml`

```yaml
name: customer-success-agent
container:
  image: ${AGENT_IMAGE}
  protocols:
    - responses
    - activity
  resources:
    cpu: 1
    memory: 2Gi
env:
  - name: FOUNDRY_PROJECT_ENDPOINT
    value: ${PROJECT_ENDPOINT}
  - name: FOUNDRY_TOOLBOX_ENDPOINT
    value: ${TOOLBOX_ENDPOINT}
  - name: AZURE_OPENAI_DEPLOYMENT
    value: gpt-4o
networking:
  vnet:
    subnet_id: ${AGENT_SUBNET_ID}
versioning:
  rollout:
    strategy: weighted
    canary_percent: 10
```

9.3 Promotion workflow (GitHub Actions)

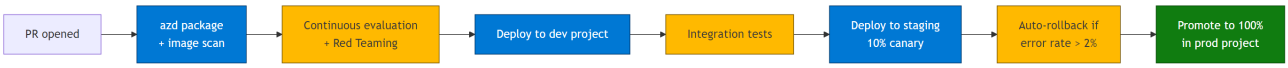

Figure 6: Diagram 6

9.4 One-command developer flow

```bash
# Bootstrap + deploy a fresh agent
azd init --template foundry-hosted-agent-quickstart
```

```
azd up

# Iterate
git commit -am "improve triage prompt"
azd deploy             # builds, pushes, creates new version

# Canary
azd agent rollout --canary 10

# Promote
azd agent promote --version v4
```

---

10 8. Security & Governance

Treat a hosted agent as **production application code**.

10.1 Identity & access (no shared service accounts)

Layer	Mechanism
Agent → Foundry models	Per-agent **Entra Agent ID** (auto-assigned at deploy)
Agent → user data	**OBO (On-Behalf-Of)** flow — auditable, revocable per user
Agent → Azure resources (Storage, KV, Cosmos…)	Manual RBAC to the agent's Entra identity
Agent → Toolbox	Bearer token via Entra; toolbox forwards via OAuth passthrough or its own MI
Tool → external SaaS	OAuth identity passthrough configured **once** in Toolbox

10.2 Network isolation

- **BYO VNet** — outbound agent traffic flows through your VNet, enabling Private Endpoints to data sources
- **Private Link** to Foundry, ACR, Key Vault, Storage, Cosmos
- No public egress required for data-plane calls

10.3 Content safety & DLP

- **Azure AI Content Safety** policies applied at the agent level (Prompt Shield, jailbreak detection, harmful content filtering)
- **DLP policies** centrally applied in Foundry Control Plane
- **Responsible AI guardrails** (system metaprompts, content filters, abuse monitoring) enforced platform-wide

10.4 Secrets

 Never put secrets in container images or env vars. Use **Foundry Key Vault connections** + Managed Identity for any secret-bearing dependency.

10.5 Third-party tool risk

If your agent calls non-Microsoft tools or models, **review what data flows out** — retention, location, governance. Microsoft makes no representation about third-party data handling. Apply DLP and Content Safety at the agent boundary.

---

11 9. Observability & Operations

Observability is GA on core capabilities and is built around **OpenTelemetry**.

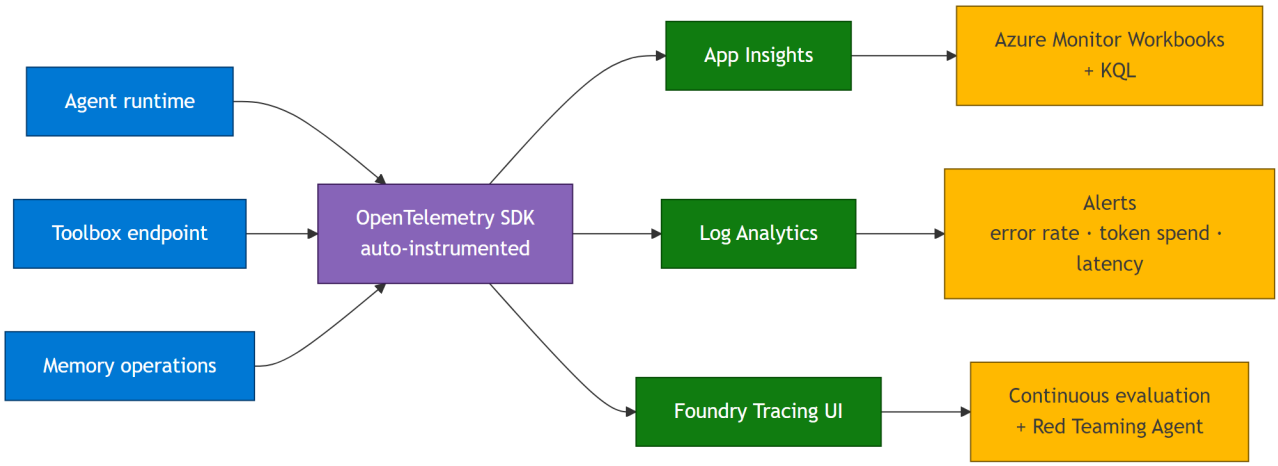

Figure 7: Diagram 7

11.1 What's in scope

Signal	Captured by	Use
End-to-end traces	OpenTelemetry auto-instrumentation across agent, toolbox, model calls	Debug latency, follow a request through a multi-agent workflow
Per-tool invocation traces	Toolbox MCP layer	Spot tool errors, slowness, governance violations
Token & cost metrics	Foundry tracing	Per-agent, per-conversation cost attribution
Continuous evaluation	Foundry Control Plane	Quality regressions caught before promotion
Adversarial coverage	AI Red Teaming Agent	Automated attack scenarios on each version

Signal	Captured by	Use
Alerts	Azure Monitor	Error spikes, token-budget exhaustion, model unavailability

11.2 Recommended baseline alerts

1. Agent version error rate > 2% over 5 min
2. Tool call P95 latency > 10 s sustained
3. Token spend per project > 80% of monthly budget
4. Memory retrieval failure rate > 1%
5. Red-teaming regression on any new version (block promotion)

---

12 10. Cost & FinOps

Hosted Agents cost economics are fundamentally different from traditional compute, **largely in your favor**.

Cost lever	Mechanism	Impact
Scale-to-zero	Compute deprovisioned after 15 min idle; you pay $0	Removes always-on baseline cost
Per-session sandbox	You pay only for active execution per session	Linear cost per real user activity
Per-agent right-sizing	`cpu` / `memory` declared per version	Avoid over-provisioning; tune by workload
Toolbox reuse	One implementation per tool, shared across agents	Removes duplicated dev cost & duplicated egress
Memory-as-a-service	Foundry-managed; you pay $0.25/1K events + retrievals	No DB to provision, scale, secure
Model selection per agent	Multi-model — pick the cheapest model that meets quality	Cost per 1K tokens varies 10× across models
Continuous eval gating	Block costly low-quality versions before promotion	Quality-cost optimization in CI/CD

12.1 Memory pricing reminder (effective June 1, 2026)

Component	Price
Short-term memory	$0.25 per 1K events stored
Long-term memory	$0.25 per 1K memories per month
Memory retrieval	$0.50 per 1K retrievals

12.2 Patterns to adopt early

- **Tag agents and conversations** with cost-center metadata for chargeback

- Set **per-project token budgets** in Azure Monitor with alerts at 80% / 100%
- Use **canary rollouts** to detect cost regressions before they hit 100% of traffic
- **Compact context** aggressively for long sessions — token cost grows quadratically without it

---

13 11. Decision Matrix — Hosted Agent vs Prompt-Based Agent

You need to...	Choose	Why
Define an agent in prompts + tool config only	**Prompt-based agent**	No-code, stays in the portal
Bring your own framework (MAF, LangGraph, SK, custom)	**Hosted Agent**	Containerized — bring any code
Accept webhooks or non-OpenAI payloads	**Hosted Agent** + Invocations	Custom payload contract
Specify CPU/memory for the sandbox	**Hosted Agent**	Resource control per version
Persist files/state across turns (`$HOME`, `/files`)	**Hosted Agent**	Per-session FS persistence
Run inside your own VNet	**Hosted Agent**	BYO VNet supported
Need per-agent Entra ID + OBO	**Hosted Agent**	Built-in; not available to prompt agents

---

14 12. Test & Simulation

Before promoting a new version, exercise the rough paths.

14.1 Inject failures at the protocol layer

```
# Force a model 429 in a test environment
import httpx, pytest
from agent_framework.testing import MockResponsesClient

mock_client = MockResponsesClient(
    responses=[httpx.Response(429, json={"error": "rate_limit_exceeded"})]
)
agent = mock_client.as_agent(name="triage", instructions="...")

with pytest.raises(RetryableError):
    await agent.run("test prompt")
```

14.2 Validate canary metrics with KQL

```
AppInsights_Traces
| where customDimensions.agent_name == "customer-success-agent"
| where customDimensions.agent_version in ("v3", "v4")
| summarize
    error_rate = countif(customDimensions.outcome == "error") * 100.0 / count(),
    p95_latency = percentile(durationMs, 95)
  by tostring(customDimensions.agent_version)
```

14.3 Adversarial coverage

Wire the **AI Red Teaming Agent** into your promotion gate. Each candidate version is exercised with attack scenarios; promotion is blocked on regression. Combine with continuous evaluation suites for quality regressions.

14.4 Chaos for downstream services

Use **Azure Chaos Studio** to inject faults in dependent services (model regional outage, Toolbox tool 503, memory store throttling) to validate that the agent degrades gracefully — surfaces a useful error to the user, retries with backoff, and emits actionable telemetry.

---

15 13. Production Readiness Checklist

15.1 Architecture

- ☐ Hosted Agent packaged as a versioned container image in **ACR**
- ☐ Two protocols at most per agent (avoid sprawl); justify each
- ☐ Stable endpoint exposed; canary slot configured
- ☐ BYO VNet enabled; outbound goes through Private Endpoints

15.2 Identity & Security

- ☐ Per-agent **Entra Agent ID** with least-privilege RBAC
- ☐ User actions flow via **OBO**, not via the agent identity
- ☐ No secrets in image or env vars; **Key Vault connection** in place
- ☐ Content Safety + Prompt Shield enabled
- ☐ DLP and Responsible AI policies applied at the project

15.3 Tools

- ☐ Toolboxes for any tool used by ≥ 2 agents
- ☐ Toolbox versioned; consumers point at the unversioned default endpoint
- ☐ OAuth passthrough or Entra Agent ID auth configured per tool
- ☐ Per-tool tracing visible in App Insights

15.4 State

- ☐ Memory store provisioned with explicit `scope` per user / tenant
- ☐ CRUD operations exposed for user data-rights requests (read / delete)
- ☐ Context compaction strategy chosen (sliding window or summarization)

15.5 Operations

- ☐ OpenTelemetry traces flowing to App Insights and Foundry tracing UI
- ☐ Continuous evaluation suite gating promotions
- ☐ Red Teaming Agent gating promotions
- ☐ Cost alerts at 80% / 100% of monthly budget per project
- ☐ Runbook for rollback (`azd agent rollout --version <prev>`)

15.6 Distribution

- ☐ Activity protocol enabled if published to Teams or M365 Copilot
- ☐ A2A endpoint exposed if other agents will delegate to this one

---

16 14. Required Personas for Industrialization

A successful agent platform involves more roles than the agent developer alone. The following RACI shows the minimum set.

Persona	Owns	Typical seat
Foundry Project Owner	Project lifecycle, RBAC at project scope	Platform team
Agent Developer	Agent code, prompts, tools, evaluation suites	Product / app team
Tools Owner	Toolbox curation, tool auth config, governance policies	Platform / integration team
Identity Engineer	Entra Agent ID, OBO flows, conditional access	Security / IAM
Network/Security Engineer	VNet, Private Endpoints, DNS, DLP	Cloud Network team
MLOps / FinOps	CI/CD, cost dashboards, alerts, model selection	Platform / FinOps
Responsible AI advisor	Content Safety policies, Red Teaming review, RAI sign-off	Compliance / Legal / Risk
Service / Product Owner	Roadmap, business metrics, channel publication	Business
End-user Support	Triage, escalation, feedback loop into evaluation	Support / Ops

---

17 15. References

17.1 Source articles (this guide synthesizes and extends them)

- Introducing the new Hosted Agents in Foundry Agent Service
- Introducing Toolboxes in Foundry
- Deploying Production-Grade Agents on Microsoft Foundry — end-to-end developer journey
- Microsoft Agent Framework v1.0 announcement
- Agent Harness in Microsoft Agent Framework
- Build AI agents with GitHub Copilot SDK and Microsoft Agent Framework
- Microsoft Foundry Toolkit for VS Code is now Generally Available

17.2 Microsoft Learn

- Hosted agents — concept docs
- Hosted agent quickstart with `azd`
- Toolbox in Foundry — how-to
- Memory in Foundry Agent Service
- Microsoft Agent Framework documentation
- Set up a Key Vault connection in Foundry

17.3 Tooling

- `azd` (Azure Developer CLI) for Foundry hosted agents
- Foundry Toolkit for VS Code
- Sample code — Toolbox + Hosted Agents:
  - Microsoft Agent Framework
  - LangGraph
  - Copilot SDK

17.4 Community

- Foundry Discord
- Foundry GitHub forum
- Microsoft Mechanics — Hosted Agents (12-min video)
- Live series — *Host your agents on Foundry*